

4/13 期中考，與 4/22 加分考

Problem 1, 2, 3, Bonus

題目目錄

UVa 11462 - Age Sort (年齡排序)	2
題目敘述	2
題解說明	2
完整程式碼	2
ZeroJudge c148 - 機器人障礙賽 (0-1 BFS)	3
題目敘述	3
題解說明	3
完整程式碼	3
ZeroJudge a825 - 小四數學(乘積最大化)	6
題目敘述	6
題解說明	6
完整程式碼	6
UVa 10326 - The Polynomial Equation	8
題目敘述	8
題解說明	8
圖的結構說明	8
逐步過程	8
係數轉移規則（核心重點）	9
完整程式碼	9

UVa 11462 - Age Sort (年齡排序)

<https://zerojudge.tw/ShowProblem?problemid=d190>

<https://vjudge.net/problem/UVA-11462>

題目敘述

給定 n 個介於 1 到 100 之間的整數（代表年齡），請將它們由小到大排序並輸出。

輸入規範：

- 多筆測資，每筆第一行為一個整數 n ($0 < n < 2000000$)。
- 接下來有 n 個整數。當 $n = 0$ 時代表輸入結束。
- 輸出時，相鄰兩個數字間以一個空白隔開，**最後一個數字後不可有空白**。

題解說明

這是一道基礎的排序題。雖然測資量 n 非常大（高達兩百萬），但我們仍可利用 C++ 強大的內建模組來解決。

1. STL 內建排序 (本文解法)：

- 使用 `<algorithm>` 中的 `sort()` 函數。其底層實作為 Introsort（混合了快速排序、堆積排序與插入排序），平均時間複雜度為 $O(n \log n)$ 。
- **輸出格式控制**：這是許多新手常踩的坑 (Presentation Error)。我們利用迴圈先輸出前 $n - 1$ 個元素並加上空白，最後一個元素 `v[n-1]` 則獨立輸出並換行。

2. 【補充】計數排序 (Counting Sort)：

- **為什麼適用？** 雖然 n 很大，但數值範圍極小（只有 1 到 100）。
- **原理**：我們可以不使用比較排序，而是宣告一個大小為 101 的陣列 `count`，每讀入一個年齡 x ，就將 `count[x] += 1`。最後從 1 遍歷到 100，按照次數將數字印出來即可。
- 這樣可以將時間複雜度降到真正的 $O(n)$ ，且不需將兩百萬個數字全部存入 `vector` 中，大幅節省記憶體。

範例追蹤：

輸入 $n = 5$ ，數值為：55, 2, 45, 13, 2

1. **讀入與儲存**：陣列狀態為 [55, 2, 45, 13, 2]
2. **執行 sort**：陣列變為 [2, 2, 13, 45, 55]
3. **格式化輸出**：
 - 前 $n - 1$ 個：輸出 2 、2 、13 、45
 - 第 n 個：輸出 55\n

完整程式碼

```
1 #include <iostream>
2 #include <algorithm>
3 #include <vector>
4 using namespace std;
5
6 int main()
7 {
8     int n;
9     while (cin >> n and n != 0)
10     {
11         vector<int> v(n);
12         for (int i = 0; i < n; ++i) { cin >> v[i]; }
13
14         sort(v.begin(), v.end(), less<int>());
15         for (int i = 0; i <= n - 2; ++i) { cout << v[i] << " "; }
16         cout << v[n-1] << "\n";
17     }
18
19     return 0;
20 }
```

ZeroJudge c148 - 機器人障礙賽 (0-1 BFS)

<https://zerojudge.tw/ShowProblem?problemid=c148>

題目敘述

在一個 $n \times m$ 的網格中，給定起點的行數 b （座標 $(0, b)$ ）與終點的行數 e （座標 $(n - 1, e)$ ），以及 k 個障礙物座標。

移動與計費規則：

1. 只能向左、向右或向下移動（不可向上回頭）。
2. **轉彎費用**：改變移動方向記為 1 次轉彎，維持原方向繼續走不計費用（即 0 次）。

請計算從起點走到終點的 **最少轉彎數**。

題解說明

這是一道經典的 **0-1 BFS (雙端隊列廣度優先搜尋)** 問題。因為圖中的邊權只有 0（不轉彎）和 1（轉彎），利用 deque 可以達到 $O(V + E)$ 的線性時間複雜度，比 Dijkstra 更高效。

1. **狀態定義**：

- 因為到達同一個座標時，面向的方向不同，後續轉彎的成本也會不同。因此必須將「方向」加入狀態中。
- 陣列 `turn[r][c][d]` 代表：到達座標 (r, c) 且當前方向為 d 的最少轉彎數。
- 方向枚舉：左 (LEFT=0)、下 (DOWN=1)、右 (RIGHT=2)。

2. **水滲透轉移邏輯**：如同水流沿著直線快速滲透，當方向改變時才會有「阻力」：

- 若目前 **朝左**：可繼續朝左（費用 0），或轉為朝下（費用 1）。
- 若目前 **朝下**：可繼續朝下（費用 0），或轉為朝左、朝右（費用 1）。
- 若目前 **朝右**：可繼續朝右（費用 0），或轉為朝下（費用 1）。

3. **雙端隊列 (Deque) 操作**：

- 若花費為 0：放至隊列 **前端** (`push_front`)，優先探索。
- 若花費為 1：放至隊列 **後端** (`push_back`)，延後探索。

0-1 BFS 權重轉移矩陣：

	轉向 左	轉向 下	轉向 右
由 左 進入	0	1	無效 (不可掉頭)
由 下 進入	1	0	1
由 右 進入	無效 (不可掉頭)	1	0

演算流程提示：起點為 $(0, b)$ ，由於是從上方進入地圖，初始狀態設定為「朝下」：`turn[0][b][DOWN] = 0`。
最終答案為取到達底層 $(n - 1, e)$ 三個方向中的最小值：

$$\text{ans} = \min(\text{turn}[n - 1][e][0], \text{turn}[n - 1][e][1], \text{turn}[n - 1][e][2])$$

完整程式碼

```
1 #include <iostream>
2 #include <algorithm>
3 #include <climits>
4 #include <array>
5 #include <vector>
6 #include <deque>
7 using namespace std;
8
```

```

9  enum Direction
10 {
11     LEFT = 0,
12     DOWN,
13     RIGHT
14 };
15
16 struct State
17 {
18     int r; // row
19     int c; // col
20     int d; // direction (0:left, 1:down, 2:right)
21
22     State(int row, int col, int dir) : r(row), c(col), d(dir)
23     {
24
25     }
26 };
27
28 int main()
29 {
30     int n, m;
31     while (cin >> n >> m)
32     {
33         int b, e;
34         cin >> b >> e;
35
36         int k;
37         cin >> k;
38
39         vector<vector<bool>> blocked(n, vector<bool>(m, false));
40         for (int i = 0; i < k; i++)
41         {
42             int x, y;
43             cin >> x >> y;
44             blocked[x][y] = true;
45         }
46
47         // turn[r][c][d]: 到達 (r,c) 且方向為 d 的最少轉彎數
48         vector<vector<array<int, 3>>> turn(n, vector<array<int, 3>>(m, {INT_MAX, INT_MAX, INT_MAX}));
49
50         // 起點 (0, b), 初始朝下
51         turn[0][b][DOWN] = 0;
52
53         deque<State> dq;
54         dq.push_back(State(0, b, 1));
55
56         while (!dq.empty())
57         {
58             State current = dq.front();
59             dq.pop_front();
60
61             int r      = current.r;
62             int c      = current.c;
63             int d      = current.d;
64             int cur_cost = turn[r][c][d];
65
66             auto Try_Move = [&](int nr, int nc, int nd, int new_cost)
67             {
68                 if ((0 <= nr and nr < n) and (0 <= nc and nc < m) and !blocked[nr][nc])
69                 {
70                     if (cur_cost + new_cost < turn[nr][nc][nd])
71                     {
72                         turn[nr][nc][nd] = cur_cost + new_cost;
73                         State next(nr, nc, nd);
74                         new_cost == 0 ? dq.push_front(next) : dq.push_back(next);
75                     }
76                 }
77             };
78
79             switch (d)
80             {
81                 case LEFT:
82                     Try_Move(r, c - 1, LEFT, 0);
83                     Try_Move(r + 1, c, DOWN, 1);
84                     break;

```

```

85         case DOWN:
86             Try_Move(r + 1, c, DOWN, 0);
87             Try_Move(r, c - 1, LEFT, 1);
88             Try_Move(r, c + 1, RIGHT, 1);
89             break;
90         case RIGHT:
91             Try_Move(r, c + 1, RIGHT, 0);
92             Try_Move(r + 1, c, DOWN, 1);
93             break;
94     }
95 }
96
97 int ans = min({turn[n-1][e][LEFT], turn[n-1][e][DOWN], turn[n-1][e][RIGHT]});
98 cout << ans << "\n";
99 }
100
101 return 0;
102 }

```

103 /* 根據當前方向決定可以走的下一步，分別有幾種選擇，如水滲透一般：

104 若目前朝左

- 105 1. 繼續朝左 (費用 0)
- 106 2. 轉成朝下 (費用 1)

107 若目前朝下

- 108 1. 繼續朝下 (費用 0)
- 109 2. 轉成朝左 (費用 1)
- 110 3. 轉成朝右 (費用 1)

111 若目前朝右

- 112 1. 繼續朝右 (費用 0)
- 113 2. 轉成朝下 (費用 1)*/

ZeroJudge a825 - 小四數學(乘積最大化)

<https://puzzling.stackexchange.com/questions/46521/maximum-result-with-digits>

<https://zerojudge.tw/ShowProblem?problemid=a825>

題目敘述

給定 m 個數字 (0-9)，將其全部使用並拆成兩個整數 a 與 b ，求 $a \times b$ 的最大可能值。

題解說明

這是一個貪心演算法問題。為了讓乘積最大，我們必須讓兩個數字的位元長度盡可能接近，且高位數的數字越大越好。

1. 策略邏輯：

- 將所有數字由大到小排序。
- 取前兩個最大的數字分別作為 a 與 b 的最高位。
- 每次將新數字接在當前數值較小的那個整數後面。

2. 數學直觀：若 $a + b$ 為定值，則當 $|a - b|$ 越小時， $a \times b$ 的值越大。

分配過程視覺化 (以 a, b 的增長為例)：

6	3	1	...	數值 a
↓	↑ ↘	↑ ↘		
5	→ 4	2	...	數值 b

邏輯推導 (以 {6, 5, 4, 3, 2, 1} 為例)：

- 排序後得 {6, 5, 4, 3, 2, 1}。
- 初始分配： $a = 6, b = 5$ 。
- 加入 4：因為 $5 < 6$ ，故 $\rightarrow b = 54$ 。
- 加入 3：因為 $6 < 54$ ，故 $\rightarrow a = 63$ 。
- 加入 2：因為 $54 < 63$ ，故 $\rightarrow b = 542$ 。
- 加入 1：因為 $63 < 542$ ，故 $\rightarrow a = 631$ 。

最終計算：

$$631 \times 542 = 342002$$

完整程式碼

```
1 #include <iostream>
2 #include <algorithm>
3 #include <vector>
4 using namespace std;
5
6 int main()
7 {
8     int m; cin >> m;
9     vector<int> v(m);
10    for (int i = 0; i < m; ++i) { cin >> v[i]; }
11
12    sort(v.begin(), v.end(), greater<int>());
13    int a = v[0];
14    int b = v[1];
15    for (int i = 2; i < m; ++i)
16    {
```

```
17     if (a <= b)
18     {
19         a *= 10;
20         a += v[i];
21     }
22     else
23     {
24         b *= 10;
25         b += v[i];
26     }
27 }
28
29 cout << a * b << "\n";
30
31 return 0;
32 }
```

UVa 10326 - The Polynomial Equation

<https://vjudge.net/problem/UVa-10326>

<https://onlinejudge.org/external/103/10326.pdf>

題目敘述

給定一個 n 次多項式方程式的所有根 (roots) $r_1, r_2, \dots, r_n$ ，請構造並輸出該多項式方程式。

輸出規範：

- 以 x 作為變數，由高次項排列至低次項，最後接 $= 0$ 。
- 若係數為 0 且非最後常數項，則不顯示該項。
- 若係數為 1，則不顯示係數（直接輸出 x^i ）。
- 常數項若為 0，必須顯示為 $+ 0$ 。

題解說明

這是一個多項式展開問題。給定根 $r_1, r_2, \dots, r_n$ ，多項式可表示為：

$$(x - r_1)(x - r_2)(x - r_3) \dots (x - r_n) = 0$$

1. 解題邏輯 (遞迴/DP)：

- 設 $P_i(x)$ 為前 i 個根組成的多項式。
- $P_{i+1}(x) = P_i(x) \times (x - r_{i+1})$ 。
- 對於 $P_{i+1}(x)$ 的第 j 次項係數 C_j ，其來源有二：
 - 前一階段的 x^{j-1} 項乘以目前的 x 。
 - 前一階段的 x^j 項乘以目前的 $(-r_{i+1})$ 。
- 轉移方程： $dp[i+1][j] = dp[i][j-1] + dp[i][j] \times (-r_{i+1})$ 。

2. 實作注意：

- 大數處理：**係數最大可達 10^{15} ，必須使用 long long。
- 格式化輸出：**需嚴格處理正負號間的空格、最高次項的正負號以及 x^1 、 x^0 的特殊顯示格式。

多項式展開邏輯圖解

這張圖用手寫方式展示如何從常數 1 開始，逐步乘上 $(x - r)$ ，來建構多項式 $(x - 1)(x - 2)(x + 3)(x - 4)(x + 5)$ 的展開式。

圖的結構說明

- 左側：**逐步的多項式乘法計算過程（手寫展開）。
- 右側：**對應每個階段的多項式係數表格（即動態規劃中的 dp 陣列）。

每一列 $dp[i]$ 代表「已經乘完前 i 個 $(x - r)$ 後，所得到的多項式之所有係數」（從常數項到最高次項）。

逐步過程

- $dp[0]$ ：初始多項式 $= 1$
- 乘上 $(x - 1) \rightarrow x - 1$
 $dp[1] = [-1, 1]$
- 乘上 $(x - 2) \rightarrow (x - 1)(x - 2) = x^2 - 3x + 2$
 $dp[2] = [2, -3, 1]$
- 乘上 $(x + 3) \rightarrow (x^2 - 3x + 2)(x + 3) = x^3 - 7x + 6$
 $dp[3] = [6, -7, 0, 1]$
- 乘上 $(x - 4) \rightarrow x^4 - 4x^3 - 7x^2 + 34x - 24$
 $dp[4] = [-24, 34, -7, -4, 1]$
- 乘上 $(x + 5) \rightarrow x^5 + x^4 - 27x^3 - x^2 + 146x - 120$
 $dp[5] = [-120, 146, -1, -27, 5, 1]$

係數轉移規則（核心重點）

當我們把目前的多項式 $P(x)$ 乘上新的 $(x - r)$ 時，新多項式每一項係數的計算方式如下：

對新係數的第 j 項 (x^j 的係數)：

$$dp[i+1][j] = dp[i][j-1] + dp[i][j] \times (-r)$$

- $dp[i][j-1]$ 代表「乘上 x 」的貢獻（係數往高一次方移動）。
- $dp[i][j] \times (-r)$ 代表「乘上 $(-r)$ 」的貢獻（原係數乘以常數）。

邊界處理：

- 最高次項 (x^{i+1}) 永遠為 1（只來自前一最高次項乘 x ）。
- 常數項 ($j = 0$) 只來自前一常數項乘 $(-r)$ 。

這正是程式碼中 $dp[i+1][j]$ 的更新邏輯。

$\backslash j$	x^0	x^1	x^2	x^3	x^4	x^5
$dp[0]$	1					
$dp[1]$	-1	1				
$dp[2]$	2	-3	1			
$dp[3]$	6	-7	0	1		
$dp[4]$	-24	34	-7	-4	1	
$dp[5]$	-120	146	-1	-27	1	1

$$1 \cdot (x - 1) = x - 1$$

$$(x - 1)(x - 2) = x^2 - 3x + 2$$

$$(x^2 - 3x + 2)(x + 3) = x^3 - 7x + 6$$

$$(x^3 - 7x + 6)(x - 4) = x^4 - 4x^3 - 7x^2 + 34x - 24$$

$$(x^4 - 4x^3 - 7x^2 + 34x - 24)(x + 5) = x^5 + x^4 - 27x^3 - x^2 + 146x - 120$$

完整程式碼

```
1 #include <iostream>
2 #include <array>
3 #include <vector>
4 using namespace std;
5 #define int long long
6
7 void Print_Term(int coef, int exp, bool is_first_term)
8 {
9     if (!is_first_term)
10     {
11         if (coef >= 0) { cout << " + "; }
12         else { cout << " - "; }
13     }
14     int val = abs(coef);
15     if (exp == 0 or val != 1) { cout << val; }
16     if (exp > 0)
17     {
18         cout << "x";
19         if (exp > 1) { cout << "^" << exp; }
20     }
21 }
22
23
24 signed main()
```

```

25 {
26     int N;
27     while (cin >> N)
28     {
29         vector<int> roots(N);
30         for (int i = 0; i < N; ++i) { cin >> roots[i]; }
31
32         vector<vector<int>> dp(N + 1, vector<int>(N + 1, 0));
33         dp[0][0] = 1; // 代表  $x^0 = 1$  (我們從常數 1 開始乘)
34
35         for (int i = 0; i < N; ++i)
36         {
37             int r = roots[i];
38             for (int j = 0; j <= i + 1; ++j)
39             {
40                 int term_x = (j > 0) ? dp[i][j - 1] : 0; // 乘以 x 的部分
41                 int term_r = dp[i][j] * -r; // 乘以 r 的部分
42
43                 dp[i + 1][j] = term_x + term_r;
44             }
45         }
46
47         // 輸出
48         bool is_first = true;
49         for (int i = N; i >= 0; --i)
50         {
51             if (i > 0 and dp[N][i] == 0) { continue; }
52             Print_Term(dp[N][i], i, is_first);
53             is_first = false;
54         }
55         cout << " = 0\n";
56     }
57
58     return 0;
59 }
60

```